Introduction:

In this project, I have fully analyzed the population changes in Finland using real-world data.
The main learning of this task was to understand how the population has changed over time and to predict future population trends.

I have used historical data from 1990 to 2025 and applied a machine learning model to predict population from 2030 to 2040.

2. Data Preparation:

First, I have loaded the dataset using pandas(pd).
This dataset usually contains population values for different regions and years.

Then, I have cleaned the data by renaming the labels so that I can make them easier to understand. I also filtered the dataset to keep only the population information and I can easily removed unnecessary parts that were not needed for the analysis.

Next, I have converted the dataset from a wide format into a long format by using the melt function. This step makes the work with easier to analyze, and visualize.

After that, I have changed the year values into integers and then converted the population values into numeric format. This ensures that the data is properly prepared for further processing and modeling.

Photos of VS Code:

```python
df = df.rename(columns={
    "Alue": "Region",
    "Tiedot": "Indicator"
})

df = df[df["Indicator"] == "Väestö 31.12."]

df = df.drop(columns=["Indicator"])

df.head()
```

✓ 0.0s Python

	Region	1990	1991	1992	1993	1994	1995	1996	1997	1998	...	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025
0	KOKO MAA	4998478	5029002	5054982	5077912	5098754	5116826	5132320	5147349	5159646	...	5503297	5513130	5517919	5525292	5533793	5548241	5563970	5603851	5635971	5652881
1	Akaa	16048	16096	16201	16142	16038	15986	15924	15840	15857	...	16923	16769	16611	16475	16391	16467	16473	16405	16387	16429
2	Alajärvi	11963	11982	12023	11992	11985	11890	11874	11803	11720	...	9899	9831	9700	9562	9419	9311	9183	9113	9078	8982
3	Alavieska	3072	3060	3053	3097	3084	3051	3054	3064	3029	...	2639	2610	2573	2519	2517	2491	2447	2437	2410	2384
4	Alavus	13785	13781	13744	13690	13623	13472	13341	13330	13299	...	11907	11713	11544	11468	11332	11197	11102	10933	10780	10634

5 rows × 37 columns

Photos of CSC Supercomputer:

information. I have also removed the "Indicator" column because it is no longer needed. Finally, I have displayed the first few rows of the dataset to check the result.

[3]:
```
df = df.rename(columns={
    "Alue": "Region",
    "Tiedot": "Indicator"
})

df = df[df["Indicator"] == "Väestö 31.12."]

df = df.drop(columns=["Indicator"])

df.head()
```

[3]:

	Region	1990	1991	1992	1993	1994	1995	1996	1997	1998	...	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025
0	KOKO MAA	4998478	5029002	5054982	5077912	5098754	5116826	5132320	5147349	5159646	...	5503297	5513130	5517919	5525292	5533793	5548241	5563970	5603851	5635971	5652881
1	Akaa	16048	16096	16201	16142	16038	15986	15924	15840	15857	...	16923	16769	16611	16475	16391	16467	16473	16405	16387	16429
2	Alajärvi	11963	11982	12023	11992	11985	11890	11874	11803	11720	...	9899	9831	9700	9562	9419	9311	9183	9113	9078	8982
3	Alavieska	3072	3060	3053	3097	3084	3051	3054	3064	3029	...	2639	2610	2573	2519	2517	2491	2447	2437	2410	2384
4	Alavus	13785	13781	13744	13690	13623	13472	13341	13330	13299	...	11907	11713	11544	11468	11332	11197	11102	10933	10780	10634

5 rows × 37 columns

Photos of VS Code:

```
df_long = df.melt(
    id_vars=["Region"],
    var_name="Year",
    value_name="Population"
)

df_long["Year"] = df_long["Year"].astype(int)
df_long["Population"] = pd.to_numeric(df_long["Population"], errors="coerce")

df_long.head()
```

[29] ✓ 0.0s

	Region	Year	Population
0	KOKO MAA	1990	4998478
1	Akaa	1990	16048
2	Alajärvi	1990	11963
3	Alavieska	1990	3072
4	Alavus	1990	13785

Photos of CSC Supercomputer:

```
[4]: df_long = df.melt(
         id_vars=["Region"],
         var_name="Year",
         value_name="Population"
     )

     df_long["Year"] = df_long["Year"].astype(int)
     df_long["Population"] = pd.to_numeric(df_long["Population"], errors="coerce")

     df_long.head()
```

[4]:

	Region	Year	Population
0	KOKO MAA	1990	4998478
1	Akaa	1990	16048
2	Alajärvi	1990	11963
3	Alavieska	1990	3072
4	Alavus	1990	13785

Comparison:

The result here I showed is from VS Code and CSC supercomputer

But if you check both environments produced the same dataset output, which confirms that the data cleaning and preprocessing steps were performed properly.

And there is no difference in values, only the execution environment is different.

CSC may provide faster performance for large-scale computations.

3. Population Trend Analysis

I have also created a line graph so that it can visualize how the population has changed over time. This graph shows the progression of population values across different years. It also helps to see clearly how the population has evolved from 1990 to 2025.

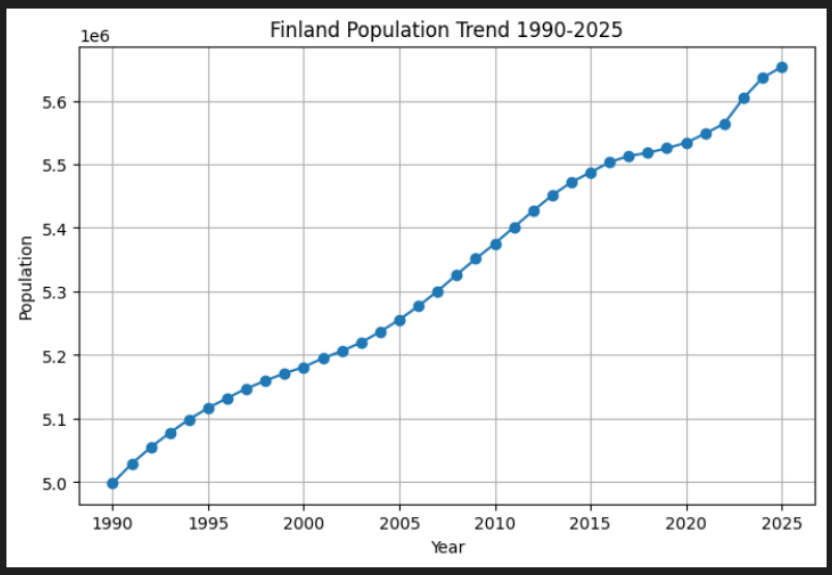
From the graph, you can see that the population has been steadily increasing over the years. This indicates a consistent growth trend over time.

Photos of VS Code:

Finally, I have displayed the graph using plt.show().

```
plt.figure(figsize=(8,5))
plt.plot(country["Year"], country["Population"], marker="o")

plt.title("Finland Population Trend 1990-2025")

plt.xlabel("Year")
plt.ylabel("Population")
plt.grid(True)
plt.show()
```

✓ 0.0s



Photos of Supercomputer:

```
[6]: plt.figure(figsize=(8,5))
plt.plot(country["Year"], country["Population"], marker="o")

plt.title("Finland Population Trend 1990-2025")

plt.xlabel("Year")
plt.ylabel("Population")
plt.grid(True)
plt.show()
```

4. Data Preparation for Machine Learning

I have prepared the data for the model by selecting the year as the input and the population as the value to use it for prediction.

After that, I have normalized the data using the average value and standard deviation. This step has helped me to improve the performance of the model and allows it to learn more efficiently.

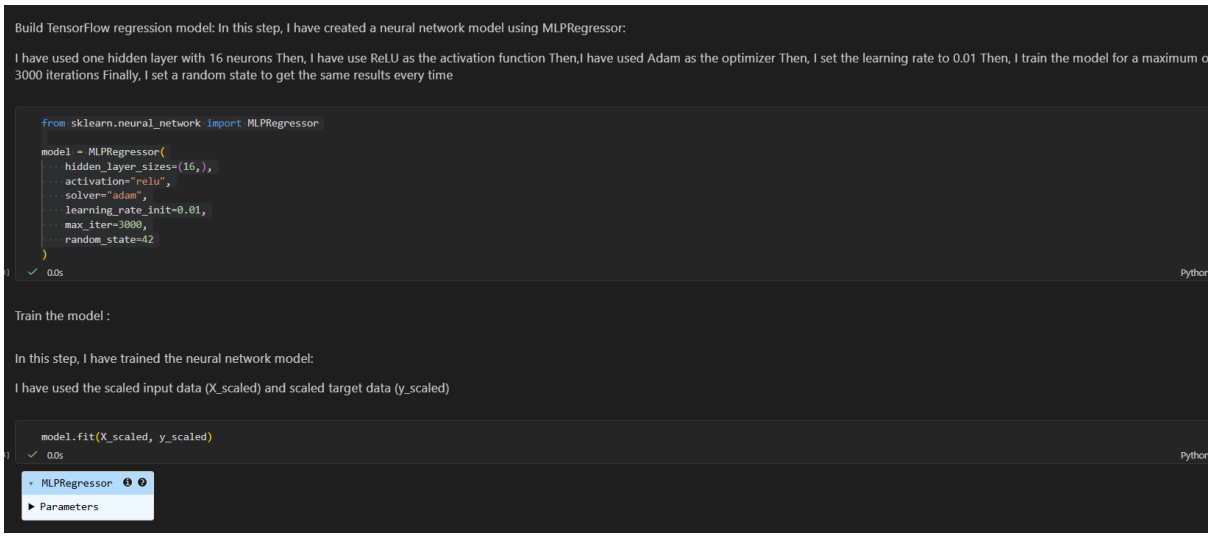
5. Model Building

I have built a neural network model so that I can learn how population changes over time.

The model is totally looks at past data and tries to find patterns. Then it uses these patterns to understand how the population is changing.

This helps the model to predict population for future years.

Photos of VS Code:



Photos of Supercomputer:

Build TensorFlow regression model: In this step, I have created a neural network model using MLPRegressor:

I have used one hidden layer with 16 neurons Then, I have use ReLU as the activation function Then,I have used Adam as the optimizer Then, I set the learning rate to 0.01 Then, I train the model for a maximum of 3000 iterations Finally, I set a random state to get the same results every time

[8]:
```
from sklearn.neural_network import MLPRegressor

model = MLPRegressor(
    hidden_layer_sizes=(16,),
    activation="relu",
    solver="adam",
    learning_rate_init=0.01,
    max_iter=3000,
    random_state=42
)
```

Train the model :

In this step, I have trained the neural network model:

I have used the scaled input data (X_scaled) and scaled target data (y_scaled)

[9]:
```
model.fit(X_scaled, y_scaled)
```

[9]:
```
MLPRegressor
▶ Parameters
```

Evaluate model on historical data : In this step, I check how well my model is performing:

I use the trained model to make predictions on the training data I calculate the loss using Mean Squared Error (MSE) I also calculate the MAE (Mean Absolute Error)

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

6. Model Training and Evaluation

I have trained the model by using the prepared data.

After the training, I have checked how well the model is working by measuring its errors.

The results show that the amount of the errors are very small. This means that the model is performing well and giving the accurate predictions.

Photos of VS Code:

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

✓ 0.0s

```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

Photos of Supercomputer:

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

0]:
```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

7. Future Prediction (2030–2040) :

I have used the model so that it can predict population for future years from 2030 to 2040 by using the analysis of previous population data.

The model used past data so that it can analyse and then predict future population values.

The result is showing that the population is expected to increase slowly over time.

Photos of VS Code:

Finally, I displayed the predicted population values for future years.

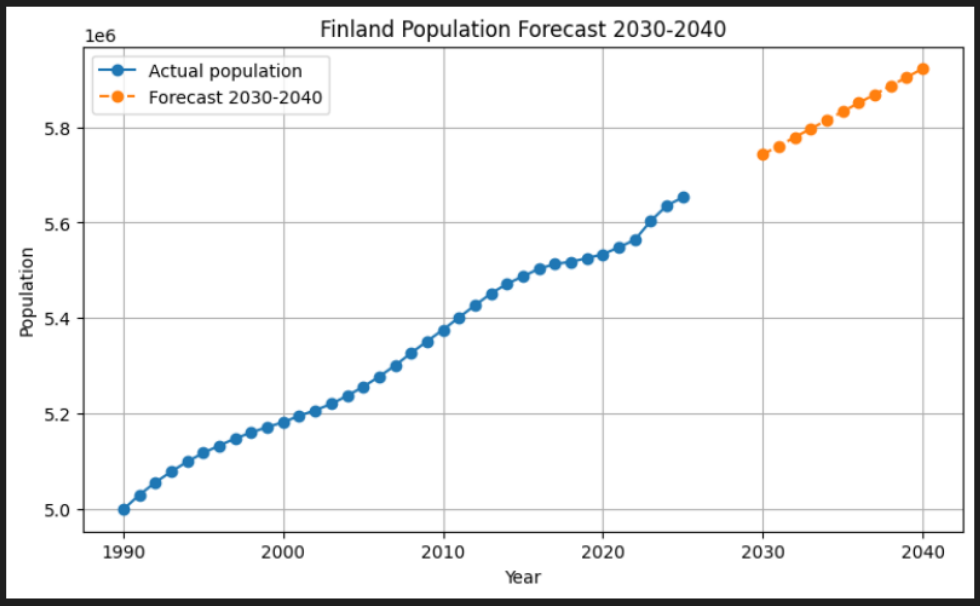
```
future_years = np.arange(2030, 2041)

future_scaled = (future_years - X_mean) / X_std
future_scaled = future_scaled.reshape(-1, 1)

future_preds_scaled = model.predict(future_scaled)

future_preds = future_preds_scaled * y_std + y_mean

forecast_df = pd.DataFrame({
    "Year": future_years,
    "Predicted Population": future_preds.astype(int)
})

forecast_df
```

✓ 0.0s

	Year	Predicted Population
0	2030	5743041
1	2031	5761029
2	2032	5779017
3	2033	5797004
4	2034	5814992
5	2035	5832980
6	2036	5850968
7	2037	5868956
8	2038	5886943
9	2039	5904931
10	2040	5922919

```python
plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```

✓ 0.2s



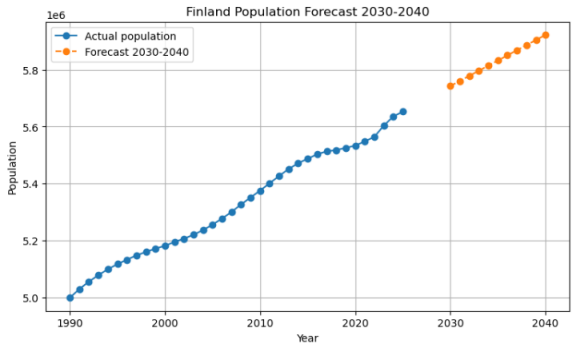
Photos of Supercomputer :

[11]:
```python
future_years = np.arange(2030, 2041)

future_scaled = (future_years - X_mean) / X_std
future_scaled = future_scaled.reshape(-1, 1)

future_preds_scaled = model.predict(future_scaled)

future_preds = future_preds_scaled * y_std + y_mean

forecast_df = pd.DataFrame({
    "Year": future_years,
    "Predicted Population": future_preds.astype(int)
})

forecast_df
```

[11]:

	Year	Predicted Population
0	2030	5743041
1	2031	5761029
2	2032	5779017
3	2033	5797004
4	2034	5814992
5	2035	5832980
6	2036	5850968
7	2037	5868956
8	2038	5886943
9	2039	5904931
10	2040	5922919

```
    forecast_df["Predicted Population"],
    marker="o",
    linestyle="--",
    label="Forecast 2030-2040"
)

plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```



8. Growth Analysis

For growth analysis, I have calculated how the population is changing over time.

Then, I looked at the average growth for different regions. Based on this, I have identified which regions are growing and then which are declining.

The results show that some regions are increasing faster, on the other side some regions are decreasing.

Growing regions may need more housing and services. On the other hand, declining regions may face challenges as there population is decreasing

Photos of VS Code:

```
Top growing regions:
Region
Liminka                  0.021624
Kempele                  0.020176
Jomala                   0.018910
Espoo                    0.018315
Pirkkala                 0.018166
TA01 Espoo-Kauniainen    0.017921
Lempäälä                 0.015632
Lemland                  0.015104
Kirkkonummi              0.014349
Vantaa                   0.014113
Name: Growth Rate, dtype: float64

Most declining regions:
Region
Rautjärvi     -0.017756
Ilomantsi     -0.017783
Siikainen     -0.018293
Posio         -0.018693
Salla         -0.018718
Rääkkylä      -0.018755
Puolanka      -0.019439
Kivijärvi     -0.019529
Hyrynsalmi    -0.020253
Rautavaara    -0.021327
Name: Growth Rate, dtype: float64
```

Photos of Supercomputer:

```
[13]: df_long = df_long.sort_values(["Region", "Year"])

df_long["Growth Rate"] = df_long.groupby("Region")["Population"].pct_change()

growth_summary = df_long.groupby("Region")["Growth Rate"].mean().sort_values(ascending=False)

print("Top growing regions:")
print(growth_summary.head(10))

print("\nMost declining regions:")
print(growth_summary.tail(10))
```

```
Top growing regions:
Region
Liminka                  0.021624
Kempele                  0.020176
Jomala                   0.018910
Espoo                    0.018315
Pirkkala                 0.018166
TA01 Espoo-Kauniainen    0.017921
Lempäälä                 0.015632
Lemland                  0.015104
Kirkkonummi              0.014349
TA43 Vantaa              0.014113
Name: Growth Rate, dtype: float64

Most declining regions:
Region
Rautjärvi    -0.017756
Ilomantsi    -0.017783
Siikainen    -0.018293
Posio        -0.018693
Salla        -0.018718
Rääkkylä     -0.018755
Puolanka     -0.019439
Kivijärvi    -0.019529
Hyrynsalmi   -0.020253
Rautavaara   -0.021327
Name: Growth Rate, dtype: float64
```

9. Anomaly Detection

For detection, I have analyzed that the population changes find some unusual patterns.

When I looked for these values, I saw that they are very different from the normal trend.

These unusual cases may be caused by many factors like migration or sudden changes in population.

For VS Code :

```python
df_long["Growth Change"] = df_long.groupby("Region")["Growth Rate"].diff()

growth_std = df_long["Growth Change"].std()
growth_mean = df_long["Growth Change"].mean()

df_long["Anomaly"] = abs(df_long["Growth Change"] - growth_mean) > 3 * growth_std

anomalies = df_long[df_long["Anomaly"] == True]

anomalies.head(20)
```

✓ 0.0s

	Region	Year	Population	Growth Rate	Growth Change	Anomaly
1143	Aura	1992	3195	0.019464	-0.039320	True
3416	Brändö	1996	534	-0.025547	-0.029210	True
3984	Brändö	1997	538	0.007491	0.033038	True
4552	Brändö	1998	525	-0.024164	-0.031654	True
6824	Brändö	2002	502	0.001996	0.027288	True
10232	Brändö	2008	518	0.015686	0.034917	True
10800	Brändö	2009	498	-0.038610	-0.054296	True
15344	Brändö	2017	452	-0.040340	-0.042467	True
15912	Brändö	2018	449	-0.006637	0.033703	True
18752	Brändö	2023	436	-0.031111	-0.033338	True
1713	Eckerö	1993	829	-0.016607	-0.042155	True
4553	Eckerö	1998	835	0.030864	0.038217	True
5689	Eckerö	2000	830	-0.018913	-0.032086	True
6257	Eckerö	2001	839	0.010843	0.029756	True
6825	Eckerö	2002	884	0.053635	0.042792	True
7393	Eckerö	2003	895	0.012443	-0.041192	True
8529	Eckerö	2005	925	0.031215	0.028981	True
12505	Eckerö	2012	960	-0.018405	-0.055520	True
15345	Eckerö	2017	948	0.021552	0.029038	True
17617	Eckerö	2021	933	-0.026096	-0.032399	True

For Supercomputer:

```
[14]: df_long["Growth Change"] = df_long.groupby("Region")["Growth Rate"].diff()

      growth_std = df_long["Growth Change"].std()
      growth_mean = df_long["Growth Change"].mean()

      df_long["Anomaly"] = abs(df_long["Growth Change"] - growth_mean) > 3 * growth_std

      anomalies = df_long[df_long["Anomaly"] == True]

      anomalies.head(20)
```

[14]:

	Region	Year	Population	Growth Rate	Growth Change	Anomaly
1143	Aura	1992	3195	0.019464	-0.039320	True
3416	Brändö	1996	534	-0.025547	-0.029210	True
3984	Brändö	1997	538	0.007491	0.033038	True
4552	Brändö	1998	525	-0.024164	-0.031654	True
6824	Brändö	2002	502	0.001996	0.027288	True
10232	Brändö	2008	518	0.015686	0.034917	True
10800	Brändö	2009	498	-0.038610	-0.054296	True
15344	Brändö	2017	452	-0.040340	-0.042467	True
15912	Brändö	2018	449	-0.006637	0.033703	True
18752	Brändö	2023	436	-0.031111	-0.033338	True
1713	Eckerö	1993	829	-0.016607	-0.042155	True
4553	Eckerö	1998	835	0.030864	0.038217	True
5689	Eckerö	2000	830	-0.018913	-0.032086	True
6257	Eckerö	2001	839	0.010843	0.029756	True
6825	Eckerö	2002	884	0.053635	0.042792	True
7393	Eckerö	2003	895	0.012443	-0.041192	True
8529	Eckerö	2005	925	0.031215	0.028981	True
12505	Eckerö	2012	960	-0.018405	-0.055520	True
15345	Eckerö	2017	948	0.021552	0.029038	True
17617	Eckerö	2021	933	-0.026096	-0.032399	True

10. Clustering Analysis :

Here, I have grouped regions into different categories based on their population and how it changes over time.

Moreover, I have created three groups so that it can represent different types of regions.

These groups helps me to understand which regions are similar and show the necessary patterns in population changes.

Photos of VS Code:

Finally, I have displayed the first few rows of the clustered data.

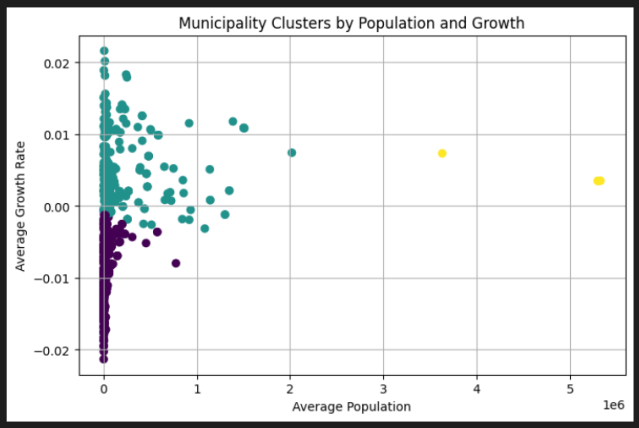
```python
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

cluster_data = df_long.groupby("Region").agg(
    avg_population=("Population", "mean"),
    avg_growth=("Growth Rate", "mean")
).dropna()

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_data)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

cluster_data["Cluster"] = kmeans.fit_predict(cluster_scaled)

cluster_data.head()
```

[40] ✓ 0.0s

Region	avg_population	avg_growth	Cluster
Akaa	16443.722222	0.000686	1
Alajärvi	10670.388889	-0.008143	0
Alavieska	2792.611111	-0.007185	0
Alavus	12500.138889	-0.007377	0
Asikkala	8464.555556	-0.003349	0

```python
plt.title("Municipality Clusters by Population and Growth")
plt.xlabel("Average Population")
plt.ylabel("Average Growth Rate")
plt.grid(True)
plt.show()
```

[41] ✓ 0.1s



Clustering Interpretation :

The clusters group regions with similar population size and growth patterns, helping to understand regional differences.

For Supercomputer:

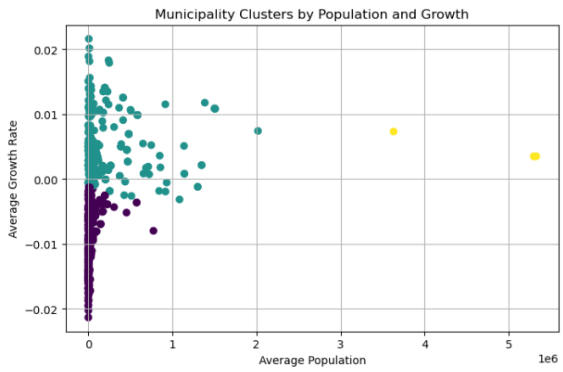
```
df_long["Anomaly"] = abs(df_long["Growth Change"] - growth_mean) > 3 * growth_std

anomalies = df_long[df_long["Anomaly"] == True]

anomalies.head(20)
```

[14]:

	Region	Year	Population	Growth Rate	Growth Change	Anomaly
1143	Aura	1992	3195	0.019464	-0.039320	True
3416	Brändö	1996	534	-0.025547	-0.029210	True
3984	Brändö	1997	538	0.007491	0.033038	True
4552	Brändö	1998	525	-0.024164	-0.031654	True
6824	Brändö	2002	502	0.001996	0.027288	True
10232	Brändö	2008	518	0.015686	0.034917	True
10800	Brändö	2009	498	-0.038610	-0.054296	True
15344	Brändö	2017	452	-0.040340	-0.042467	True
15912	Brändö	2018	449	-0.006637	0.033703	True
18752	Brändö	2023	436	-0.031111	-0.033338	True
1713	Eckerö	1993	829	-0.016607	-0.042155	True
4553	Eckerö	1998	835	0.030864	0.038217	True
5689	Eckerö	2000	830	-0.018913	-0.032086	True
6257	Eckerö	2001	839	0.010843	0.029756	True
6825	Eckerö	2002	884	0.053635	0.042792	True
7393	Eckerö	2003	895	0.012443	-0.041192	True
8529	Eckerö	2005	925	0.031215	0.028981	True
12505	Eckerö	2012	960	-0.018405	-0.055520	True
15345	Eckerö	2017	948	0.021552	0.029038	True
17617	Eckerö	2021	933	-0.026096	-0.032399	True

[16]:

```
plt.figure(figsize=(8,5))

plt.scatter(
    cluster_data["avg_population"],
    cluster_data["avg_growth"],
    c=cluster_data["Cluster"]
)

plt.title("Municipality Clusters by Population and Growth")
plt.xlabel("Average Population")
plt.ylabel("Average Growth Rate")
plt.grid(True)
plt.show()
```



11. Comparison: VS Code vs CSC :

I have executed the same notebook in both VS Code and CSC supercomputer environments.

The results from both environments are very similar:

- The population graphs are identical

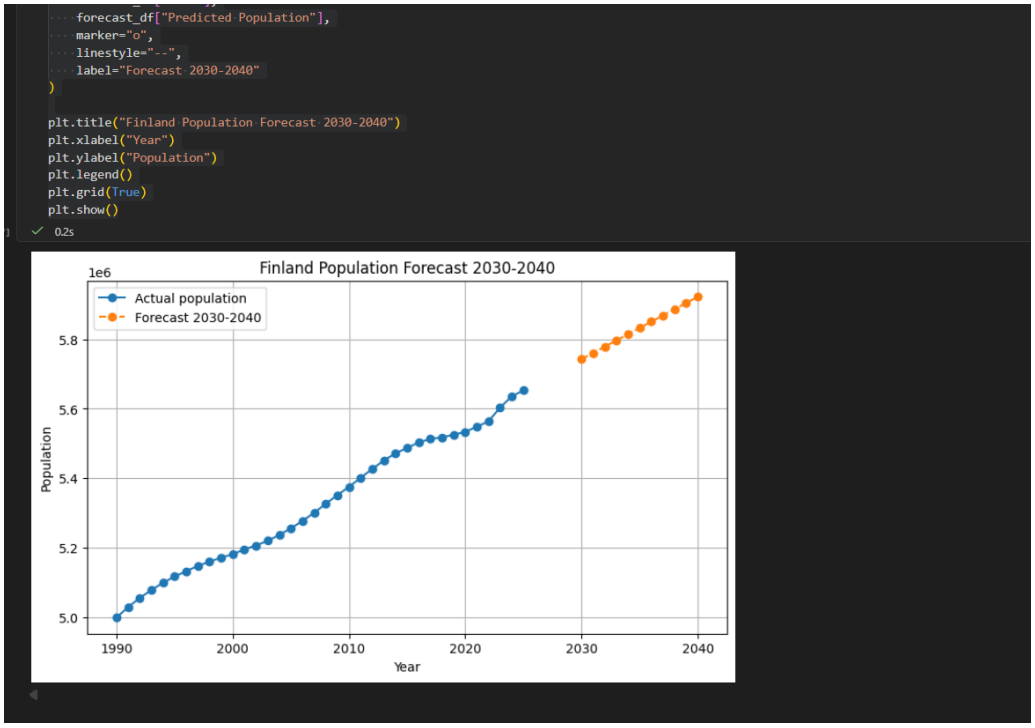
- The predicted values are almost the same
- The loss and MAE values are also similar

This shows that the model produces consistent results.

However, there are some differences:

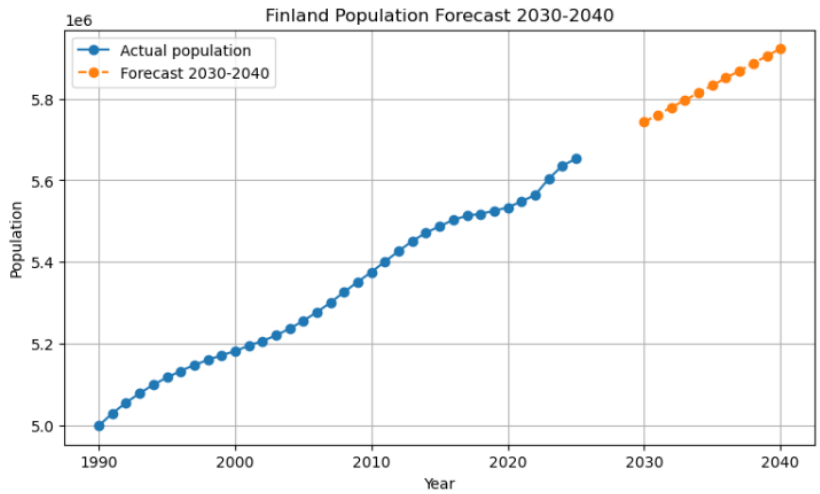
- CSC runs the code faster
- CSC is better for large datasets
- VS Code is easier to use for development

VS Code Forecast graph ;



Supercomputer Forecast Graph :

```
    marker="o",
    linestyle="--",
    label="Forecast 2030-2040"
)

plt.title("Finland Population Forecast 2030-2040")
plt.xlabel("Year")
plt.ylabel("Population")
plt.legend()
plt.grid(True)
plt.show()
```



Analyze population growth by region : In this step, I have prepared the data to analyze growth:

I have sorted the data by Region and Year I have calculated the growth rate of population for each region using percentage change

VS Code Loss/MAE:

Results : The loss shows how much error the model has (lower is better) The MAE shows the average difference between predicted and actual values

```
train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

[5] ✓ 0.0s

```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

CSC Loss/MAE :

```
[10]: train_preds = model.predict(X_scaled)

loss = np.mean((train_preds - y_scaled) ** 2)
mae = np.mean(np.abs(train_preds - y_scaled))

print("Loss:", loss)
print("MAE (scaled):", mae)
```

```
Loss: 0.01707506806203566
MAE (scaled): 0.1031003282584641
```

Conclusion

In this project, I havesuccessfully analyzed population trends in Finland and built a machine learning model which predict future values.

The results shows that:

- Population is increasing steadily
- Future population is expected to grow gradually
- Some regions grow faster than others
- Some regions are declining